

Reviewer Pack — Minimal Rank of Noncrossing Partitions Bounds Resolution Pro...

Entry 3d3a15569524 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 11:24:17 UTC

Minimal Rank of Noncrossing Partitions Bounds Resolution Proof Length for Tseitin Formulas

Entry ID: 3d3a15569524 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 11:24:17 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Noncrossing Partitions) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For a given Tseitin formula F on n variables, if the graph associated with F has minimal rank equal to $\nu(G)$, then the length of any resolution proof for F is at least $2^{(\Omega(\nu(G)))}$.’, ‘Moreover, there exists a polynomial-time computable function ν that assigns a noncrossing partition number to each graph G such that $\nu(G) = O(1)$ for all expanders.’]

Rationale (proposer’s reasoning):

[‘Noncrossing partitions have been used to analyze the structure of graphs in combinatorial geometry. This conjecture explores the potential of using this concept to derive lower bounds on resolution proof length, which is a key complexity measure.’, ‘The connection between noncrossing partitions and resolution proof length may reveal new insights into the difficulty of satisfiability problems, potentially leading to new proof techniques or complexity separations.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2683722428031a7e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Tseitin formula F on n variables, if minimal rank $\nu(G) = \nu$, then resolution proof length $L(F)$ is at least $2^{(\Omega(\nu))}$ for all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	UNCERTAIN	0.95	UNCERTAIN	HITS
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncrossing partitions' AND 'resolution proof length' AND Tseitin formulas' - 'minimal rank' AND 'graph complexity' AND 'Tseitin formula resolution' - ' $v(G)$ ' AND 'polynomial-time computable function' AND 'noncrossing partition number'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_formula(n):
        if n < 2:
            return [], []

        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []

        # Create OR clauses
        for i in range(1, n+1):
            clause = [variables[i-1]]
            for j in range(i):
                clause.append(f'~{variables[j]}')
            clauses.append(clause)

        # Create AND clauses
        for i in range(n):
            for j in range(i+1, n):
                clauses.append([f'~{variables[i]}', f'~{variables[j]}', variables[i+j]])

        return variables, clauses

    def graph_from_clauses(clauses):
        graph = {}
        for clause in clauses:
            for literal in clause:
                if literal.startswith('~'):

```

```

        var = literal[1:]
    else:
        var = literal
    if var not in graph:
        graph[var] = set()
    for other_var in clause:
        if other_var != literal and other_var not in graph[var]:
            graph[var].add(other_var)

    return graph

def min_rank(graph):
    n = len(graph)
    matrix = [[0]*n for _ in range(n)]

    # Create adjacency matrix
    for i, neighbors in enumerate(graph.values()):
        for neighbor in neighbors:
            j = variables.index(neighbor)
            matrix[i][j] = 1

    # Gaussian elimination to find rank
    rank = n
    for i in range(n):
        if matrix[i][i] == 0:
            found_pivot = False
            for j in range(i+1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    found_pivot = True
                    break
            if not found_pivot:
                rank -= 1
                continue

    # Eliminate other rows
    for j in range(n):
        if i == j:
            continue
        factor = Fraction(matrix[j][i], matrix[i][i])
        for k in range(n):
            matrix[j][k] -= factor * matrix[i][k]

    return rank

def resolution_length(clauses, variables):
    stack = []
    seen = set()

    def resolve(l1, l2):
        if l1.startswith('~'):
            var1 = l1[1:]
        else:
            var1 = l1
        if l2.startswith('~'):
            var2 = l2[1:]
        else:

```

```

        var2 = l2

    if var1 == var2:
        return None
    elif var1 in seen or var2 in seen:
        return None
    else:
        seen.add(var1)
        seen.add(var2)
        stack.append(l1)
        stack.append(l2)
        return len(stack)

for clause in clauses:
    if not any(l.startswith('~') and l[1:] in variables for l in clause):
        continue

    for literal in clause:
        if literal.startswith('~'):
            var = literal[1:]
        else:
            var = literal

        if var in seen:
            continue

        found_resolvent = False
        for other_clause in clauses:
            if any(l.startswith('~') and l[1:] == var for l in other_clause):
                resolvent_length = resolve(literal, f'~{var}')
                if resolvent_length is not None:
                    return resolvent_length
                found_resolvent = True

        if not found_resolvent:
            return 1

    return len(stack)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    variables, clauses = tseitin_formula(n)
    graph = graph_from_clauses(clauses)
    v_G = min_rank(graph)

    if v_G == 0:
        return {
            "metric_name": "resolution_length",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

```

```

length = resolution_length(clauses, variables)
results.append((v_G, length))

if not results:
    return {
        "metric_name": "resolution_length",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "no_results"
    }

v_G_values = [v for v, _ in results]
lengths = [length for _, length in results]

def spearman_correlation(x, y):
    n = len(x)
    rank_x = {x[i]: i+1 for i in range(n)}
    rank_y = {y[i]: i+1 for i in range(n)}

    sum_d_squared = sum((rank_x[x[i]] - rank_y[y[i]])**2 for i in range(n))
    return 1 - (6 * sum_d_squared) / (n * (n**2 - 1))

correlation = spearman_correlation(v_G_values, lengths)

return {
    "metric_name": "spearman_correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "conjecture_holds": correlation >= 0.95,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if "metric_value" in trial_result and trial_result["metric_value"] is not None:
            results.append(trial_result["metric_value"])

    if not results:
        print("RESULT: INCONCLUSIVE no_results")
    else:
        mean = sum(results) / len(results)
        std_dev = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
        support_fraction = sum(1 for r in results if r >= 0.95) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
        elif any(r < 0.95 for r in results):

```

```

        first_failing_seed = seeds[results.index(min(r for r in results if r < 0.95))]
        print(f"RESULT: FALSIFIED counterexample='correlation_too_low' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29f34197.py", line 199, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29f34197.py", line 146, in run_trial
    variables, clauses = tseitin_formula(n)
                        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29f34197.py", line 38, in tseitin_formula
    clauses.append([f'~{variables[i]}', f'~{variables[j]}', variables[i+j]])
                                     ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with appropriate error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12669
2	preregistration	ollama_remote	glm4:latest	0	0	5930
3	novelty	ollama_remote	glm4:latest	0	0	4840
4	novelty	ollama_remote	glm4:latest	0	0	10457
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42684
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13950
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18738
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18239
9	judge	ollama_remote	glm4:latest	0	0	11546

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 139053 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/3d3a15569524.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3d3a15569524.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3d3a15569524.tar.gz (if generated)